Presentation Transcript

Research Agent – an LLM-powered planning agent for academic literature search

Intelligent Agents (PCOM7E), University of Essex Online · Individual Project Presentation, October 2026

Narration for the 10-slide deck Research_Agent_Presentation.pptx, approximately 2,399 words. Each section is the spoken text for one slide.

Slide 1 – Research Agent

Hello, and thanks for watching. This is my individual project for the Intelligent Agents module. Earlier in the module our group designed a research agent on paper. My job here was to build it, test it, and then, once that design had been marked, act on the feedback. The system takes a research question and turns it into an evidence brief that a researcher can check line by line. In this presentation I'll cover what the design asked for, how I built it, a real run, the testing, what went wrong, what I changed after the tutor's feedback, and where it still falls short. The code, the tests, the logs and the README are all in the GitHub repository on the slide.

Slide 2 – The brief, and the design I was implementing

The starting point was our Group D design proposal. The design pinned down a lot before any code existed: three specialised agents and a coordinator that is deliberately not an agent; two human reviews; every DOI checked against Crossref; one automatic retry per sub-question and one scope revision per run; and a brief that has to say what it could not cover.

All of that comes from one fear. Ask a language model for literature and it will give you plausible papers that don't exist - Walters and Wilder measured exactly that for ChatGPT in 2023. So in this system the model never writes a reference. It turns questions into search queries, scores relevance, summarises abstracts and draws themes from those summaries - nothing more. Anything with an exact answer - de-duplication, DOI checks, the reference list - is plain code.

My task was to build that, test it, and show it working. The feedback on the design reached me at the end of September, with three points: justify the choice of model, set measurable quality targets, and use more than one literature source with automatic failover.

Slide 3 – Architecture: three agents, one coordinator

The architecture follows the class diagram from our proposal. The researcher talks to one boundary class for both reviews. Behind it, the orchestrator runs the steps in a fixed order, enforces the limits, and does the exact work itself. The Planning Agent uses the language model to turn the question into sub-questions and queries, and to rewrite a query when a search comes up short. The Retrieval Agent has no model at all: it searches two sources, OpenAlex and Semantic Scholar, for every sub-question, merges what comes back, and decides, by counting, whether it got enough usable records. The Evaluation and Synthesis Agent uses the model for judgement: scoring each paper's relevance with a reason, and later writing notes, themes and gaps from whatever was approved.

Two choices here rest on the literature. Each agent has one goal of its own, the goal-directed behaviour Wooldridge and Jennings ask of an agent, and the split into roles follows the Gaia method. To be exact, only the Retrieval Agent checks its own goal; the plan is judged by the researcher, and the number of relevant papers by the orchestrator. And there is one coordinator on purpose: Kim and colleagues reported this year that designs without central verification pass errors on more strongly.

Every model reply is checked against a schema, repaired once if needed, or the run stops. The numbers on the right are the limits the agents work within: the first three come from the design, the relevance cut-off was chosen during the build, and all four live in one configuration file.

Slide 4 – How one run flows

A run starts with one command: python main dot py, and the question. The question is planned into three to five sub-questions with search queries, and that plan goes to the researcher at Review 1: approve, or say what to change. Each sub-question is then searched on both sources, twenty records from each. The results are merged and de-duplicated by DOI, every DOI is looked up in Crossref, and

each paper is scored one to five from its abstract, with a reason. Review 2 shows every paper that reached the cut-off and offers three choices: approve, drop papers by number, or reject the scope and go back to planning. After approval the agent writes a note per paper, then themes and gaps, and the orchestrator adds limitations computed from the run and a reference list built from the records.

If a search falls short - too few usable records, or too few relevant papers after scoring - the Planning Agent rewrites the query once, and only once. And if a source fails after its retries, the Retrieval Agent skips it for that query and tries it again on the next.

Slide 5 – Demo 1 - planning and the first review

This is the first review point, on the question 'What approaches reduce hallucination in LLM-based autonomous agents'. The screenshot is from the fifteenth of September, on the full Flash model: an interpretation and four sub-questions, each with a query. Nothing has been searched at this point, so the plan can be revised as often as needed; each revision is one more model request.

When I moved to the Lite model, the same question produced a different plan, the one in the first box, and the run of the thirtieth on the next two slides used it. Two models, two plans for one question: that is why a person approves the plan.

It was on that Lite plan that I tested a revision. I asked to merge sub-questions three and four into one about evaluation, and the next plan came back with exactly three, one of them on evaluation. Reading the code again on the first of October, I found that the revision prompt had never contained the plan it was revising. The model had followed my words, not the numbers: the old third sub-question, on self-correction and memory, was simply gone. The plan now travels with the feedback. On the second of October I ran the same request again: this time the first two sub-questions came back word for word, and the third and fourth were combined into one, on self-correction and evaluation. Only the memory part was lost.

Slide 6 – Demo 2 - two sources, verify, score, retry once

This is the run of the thirtieth of September, after the second source was added. Each sub-question went to both sources. OpenAlex answered all five searches, a hundred records. Semantic Scholar, used without a key, refused three of the five with 'too many requests'. Each time, after four retries, the Retrieval Agent logged the failure and carried on with OpenAlex. That is the failover the feedback asked for: the run didn't die, and the brief says how many searches each source answered.

A hundred and forty records came back; a hundred and seventeen were unique. Seventy-eight DOIs were found in Crossref; the rest are mostly arXiv preprints, which register with DataCite instead, so they're flagged but kept. Every paper was then scored with a reason, and seventy-seven reached the cut-off.

The code at the top is the part of the orchestrator that decides on a retry. There are two checks: the Retrieval Agent's own count of usable records, and, after scoring, a count of the relevant ones. The comment says why the second exists: a search can return plenty of abstracts of which only one answers the question. Both checks share a single retry.

The lower panel is that retry doing its job. Sub-question 4, on evaluation, produced eight new records with only two relevant. The orchestrator asked for one reformulation. This time both sources answered the new query: thirty-three new records, twenty of them relevant, so that sub-question finished with twenty-two papers instead of two. A week earlier, with one source and the same plan and model, the same retry had taken it from two papers to six.

Slide 7 – Demo 3 - the second review and the brief

Review 2 lists every paper that passed, grouped and numbered, with score and reason. In the functional tests I tried all three choices - approve, drop, reject; a second reject was refused because that budget was used.

After approval the seventy-seven papers were summarised in ten batches, and the themes and gaps took one more call. The note for paper seventy-six describes a practitioner audit built on forty-five

recorded incidents; I checked, and that figure is in the paper's abstract - and that paper came from Semantic Scholar only, so the single-source system would never have seen it. Each theme cites paper numbers, and the schema rejects a theme that cites a number not in the list. The limitations are computed by the orchestrator, not written by the model, and they include what each source contributed. The reference list comes straight from the records' metadata. The model cites papers by number; it never writes a reference.

Slide 8 – Testing: three layers of evidence

Testing happened at three levels. A hundred and thirteen unit tests run without any network; they cover every agent and every check, and the slide lists them. Sixteen functional tests were run live, each with its steps, the expected result and what actually happened. Eleven were run during development, from a missing API key to the two-source run with real failover; five more on the second of October, on the submitted code. All sixteen pass, two with a qualification: the early plan revision, as I said, and the constraint test, which showed that a constraint steers the search but does not filter the results. And a remediation log with twenty-one entries records every problem I hit, its cause, the change, and how it was checked.

The feedback also asked for clear targets and the dataset to test them on. The README now states them: precision of at least zero point eight at the cut-off and recall of at least zero point six, measured on a gold set - the hundred and twenty or so records of this question judged by two people, and the studies included in a published review. That set has not been built, so I claim no result for either. Citation accuracy holds by construction, because a fabricated citation cannot reach the brief. De-duplication misses its target: four papers appear twice under different DOIs. And the proposal planned a varied set of test questions. For nearly all of the work I used one; on the last day I ran three others - telemedicine, food waste in retail, and a deliberately vague one. Each produced a complete brief, but no person has judged those briefs. For the telemedicine run I did get a second opinion: the full Gemini Flash model judged all sixty-five papers from the same titles and abstracts, without seeing the agent's scores. Of the thirty-five kept papers it answered clearly on, it would keep twenty-seven; across all its

clear answers, kappa was zero point six five. Every kept paper it would not keep sat exactly at the cut-off. That is agreement between two models, not precision.

Slide 9 – What went wrong, and what changed

Most of the problems on this slide came from outside the code. The very first Gemini call returned a 404: the model I had configured had been retired by the provider. The name was in configuration, so the fix was a change of setting. Then runs died on a 503 and on a read timeout; that produced the retry for transient errors, while a wrong request still fails at once. Then came 429s: the free tier for that model allows twenty requests a day, retries included. The client now stops at once on a daily quota, and I switched to a Flash-Lite model with five hundred a day - again only a setting. On the twenty-third OpenAlex started refusing requests - keyless use is now capped - so a free key went into configuration and the retry logic into one shared policy for the scholarly APIs.

None of this needed a rewrite: the settings sit in one module and each service behind one small client. The commit history starts on the ninth of September, roughly one commit per stage. The last ten entries in the log are of a different kind. On the first of October I read the code and the documents against each other and found a rejected model reply being kept in the cache, a title check that could not fail for titles in other scripts, the revision prompt I mentioned, and an error message that could have written an API key into the log. Those are fixed and unit-tested. The run of the thirtieth, replayed offline from the saved replies, produced the same papers, scores, notes and themes, and the five live tests of the second of October ran on the corrected code.

Slide 10 – Against the design, and honestly

To finish, a comparison with the design. Where it matters, the implementation matches the design: the agents, the orchestrator, both reviews, both limits, the Crossref check and the brief. The README lists every departure; these are the main ones. The proposal put Semantic Scholar first with OpenAlex as the fallback, and I built on OpenAlex alone until the last stage, because it needed no key when I started. Checking and scoring run one sub-question at a time, with a second retry trigger after scoring.

Crossref compares the title, not all the metadata. And a second scope rejection is refused instead of stopping the run.

The feedback had three points, and each has an answer. The model choice is now justified with the provider's own figures - context size against the size of my prompts, structured output, free-tier quota, price and benchmarks - with two caveats: function calling is not used at all, because the orchestrator calls the tools, and no benchmark measures abstract screening. The targets and their datasets are stated, as I said. And the third point was built: two sources for every search, failover between them, seventy-seven unique papers becoming a hundred and seventeen on the same plan and model.

The limits: abstracts only, twenty per query per source. Without a key, Semantic Scholar refused every live search on the second of October, so the new searches of those runs rested on OpenAlex alone. The Lite model kept forty-two of seventy-seven papers where the full Flash model kept twenty-six of seventy-eight - each on its own plan - and the second opinion disagreed only around the cut-off, so the cut-off needs calibrating per model. One professional point: on the free tier the provider may use prompts to improve its products, so a confidential question needs a paid key. Next would be a DataCite lookup for the preprints, the gold set, and full text where it is open. Everything is in the repository; the references are in the README and the transcript. Thank you for listening.

References – sources cited on the slides and in the narration

The letters after 2026 are those of the README's reference list, where Google (2026a) and Google DeepMind (2026b) also appear.

Google (2026b) *Gemini 3.5 Flash-Lite*. Gemini API documentation. Available at: https://ai.google.dev/gemini-api/docs/models/gemini-3.5-flash-lite (Accessed: 30 September 2026).

Google (2026c) *Gemini Developer API pricing*. Gemini API documentation. Available at: https://ai.google.dev/gemini-api/docs/pricing (Accessed: 30 September 2026).

Google (2026d) *Rate limits*. Gemini API documentation. Available at: https://ai.google.dev/gemini-api/docs/rate-limits (Accessed: 30 September 2026).

Google DeepMind (2026a) *Gemini 3.5 Flash-Lite*. Available at: https://deepmind.google/models/gemini/flash-lite/ (Accessed: 1 October 2026).

Group D (2026) *Project report*. Team design proposal (Assessment 1), Intelligent Agents, University of Essex Online. Unpublished.

Hong, S. et al. (2024) 'MetaGPT: meta programming for a multi-agent collaborative framework', *The Twelfth International Conference on Learning Representations*, Vienna, Austria, 7–11 May 2024. OpenReview.net. Available at: https://openreview.net/forum?id=VtmBAGCN7o (Accessed: 24 August 2026).

Kim, Y. et al. (2026) 'Capable language models can outgrow the benefits of collaboration', *Nature Machine Intelligence*, 8(7), pp. 1157–1172. Available at: https://doi.org/10.1038/s42256-026-01268-y

Walters, W.H. and Wilder, E.I. (2023) 'Fabrication and errors in the bibliographic citations generated by ChatGPT', *Scientific Reports*, 13(1), article 14045. Available at: https://doi.org/10.1038/s41598-023-41032-5

Wooldridge, M. and Jennings, N.R. (1995) 'Intelligent agents: theory and practice', *The Knowledge Engineering Review*, 10(2), pp. 115–152. Available at: https://doi.org/10.1017/S0269888900008122

Wooldridge, M., Jennings, N.R. and Kinny, D. (2000) 'The Gaia methodology for agent-oriented analysis and design', *Autonomous Agents and Multi-Agent Systems*, 3(3), pp. 285–312. Available at: https://doi.org/10.1023/A:1010071910869

Zhu, L. et al. (2026) 'Designing meaningful human oversight in AI', *AI and Ethics*, 6(3), article 286. Available at: https://doi.org/10.1007/s43681-026-01147-7